Audit interne
MyToolsApp mobile v2

Application mobile partenaires garages
React Native • Expo SDK 54 • EAS Build

Version 2.0.1 • 26 mars 2026

ÉQUIPE ET COMMANDITAIRE

Rédacteur	**Riad BELMAHI**	CTO — Chief Technology Officer
Direction	**Iliass MEGAIZ**	CEO — Chief Executive Officer
Commanditaire	**Paf Invest Limited**	Groupe investisseur
Président	**Naime BELAMIRI**	Président — Paf Invest Limited

STATUT	EXPO-DOCTOR	EAS BUILD	PLATEFORMES
CONFIDENTIEL	**17/17**	**Configuré**	**iOS + Android**

Document confidentiel — Usage interne uniquement — © 2026 MyTools Group / Paf Invest Limited

Table des matières

- Vue d'ensemble
- Architecture générale
- Stack technologique
- Structure du projet
- Frontend — Routing et navigation
- Authentification et sécurité
- Gestion d'état et données
- API Backend — Proxy Express
- Intégration Firebase
- Base de données locale (PostgreSQL)
- Notifications push
- Build et déploiement (EAS)
- Configuration des environnements
- Composants partagés
- Points d'attention pour l'audit

---

1. Vue d'ensemble

MyToolsApp est une application mobile React Native (Expo SDK 54) destinée aux garages partenaires de MyTools Group. Elle couvre deux profils utilisateurs distincts sur une interface unifiée :

Profil	Rôles concernés	Accès
Admin / Employé	admin, super_admin, root, employe, manager	Gestion devis, factures, réservations, clients, services
Client	client, tout autre rôle	Consultation devis/factures, réservations, messagerie, historique

L'application est un client mobile d'un SaaS externe (saas2.mytoolsgroup.eu) auquel elle se connecte via un proxy Express hébergé sur Replit. L'app ne contient aucune logique métier propre — elle orchestre des appels vers l'API SaaS externe et expose localement une couche de persistence complémentaire (photos, montants, logs).

---

2. Architecture générale

```
□□□
```

```
□ □ Login     □ □ Admin tabs □ □ Client tabs   □ □
□ □ Register   □ □ Devis,Fact. □ □ Devis,Fact.   □ □
□ □ Social Auth □ □ Réservations□ □ Réservations   □ □
□ □□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□□□ □
□□□□□□□□□□□□□□□□□□□
```

Technologie	Version	Usage
Expo SDK	54.0.33	Framework React Native géré
React Native	0.81.5	Moteur UI natif
React	19.1.0	Bibliothèque UI
expo-router	6.0.23	Routing basé sur le système de fichiers
@tanstack/react-query	5.83.0	Gestion du state serveur et cache
react-native-reanimated	4.1.1	Animations natives
react-native-worklets	0.5.1	Runtime worklets (peer dep Reanimated v4)
react-native-gesture-handler	2.28.0	Gestion des gestes tactiles
react-native-keyboard-controller	1.18.5	Gestion clavier (iOS/Android)
firebase	12.11.0	Auth sociale côté client
expo-secure-store	15.0.8	Stockage sécurisé tokens (Keychain/Keystore)
expo-notifications	0.32.16	Notifications push
expo-camera / expo-image-picker	17 / 17	Photos véhicules/pièces
expo-local-authentication	17.0.8	Authentification biométrique
expo-glass-effect	0.1.9	Tab bar verre liquide (iOS 26+)
@google/genai	1.41.0	IA Gemini (OCR, analyse)

Backend proxy

Technologie	Version	Usage
Express	5.0.1	Framework HTTP
Node.js / tsx	— / 4.20.6	Runtime TypeScript
firebase-admin	13.7.0	Vérification tokens Firebase server-side
pg (node-postgres)	8.16.3	Client PostgreSQL
Busboy	—	Parse multipart/form-data (upload photos)
jsonwebtoken	9.0.3	Gestion JWT
zod	3.25.76	Validation des schémas

Build & déploiement

Outil	Version	Usage
EAS CLI	18.4.0	Build iOS/Android managé
EAS Build	—	CI/CD cloud (expo.dev)
EXPO_TOKEN	Secret Replit	Auth EAS non-interactive

4. Structure du projet

```
mytoolsapp/
□□□ app/                    # Routing Expo Router (file-based)
□   □□□ _layout.tsx           # Racine : providers, fonts, splash
□   □□□ index.tsx             # Redirect vers auth ou tabs selon rôle
□   □□□ onboarding.tsx         # Écran d'onboarding initial
□   □□□ consent.tsx           # Consentements RGPD
□   □□□ legal.tsx             # Mentions légales
□   □□□ privacy.tsx           # Politique de confidentialité
□   □□□ support.tsx           # Support public
□   □□□ (auth)/               # Groupe non-authentifié
□   □   □□□ _layout.tsx
□   □   □□□ login.tsx         # Connexion email/password + sociale
□   □   □□□ register.tsx      # Inscription
□   □   □□□ forgot-password.tsx  # Réinitialisation mot de passe
□   □□□ (admin)/              # Groupe admin/employé
□   □   □□□ _layout.tsx       # Stack admin
□   □   □□□ (tabs)/           # Navigation tabs admin
□   □   □   □□□ _layout.tsx
□   □   □   □□□ index.tsx     # Tableau de bord
□   □   □   □□□ clients.tsx   # Liste clients
□   □   □   □□□ quotes.tsx    # Liste devis
□   □   □   □□□ invoices.tsx  # Liste factures
□   □   □   □□□ reservations.tsx  # Liste réservations
□   □   □   □□□ create.tsx    # Création rapide
□   □   □   □□□ more.tsx      # Menu étendu
□   □   □   □□□ settings.tsx  # Paramètres admin
□   □   □□□ quote-create.tsx  # Création/édition devis
□   □   □□□ quote-detail.tsx  # Détail devis
□   □   □□□ invoice-create.tsx  # Création/édition facture
□   □   □□□ invoice-detail.tsx  # Détail facture
□   □   □□□ reservation-create.tsx
□   □   □□□ reservation-detail.tsx
□   □   □□□ client-form.tsx   # Formulaire client
□   □   □□□ service-create.tsx  # Création service
□   □   □□□ services-list.tsx  # Liste services
□   □   □□□ users.tsx         # Gestion utilisateurs
□   □   □□□ notifications.tsx  # Centre notifications admin
□   □   □□□ logs.tsx          # Logs activité
□   □   □□□ admin-logs.tsx    # Logs techniques
□   □   □□□ support-history.tsx  # Historique support
□   □   □□□ guide.tsx         # Guide utilisateur
□   □   □□□ delete-account.tsx  # Suppression compte
□   □□□ (main)/               # Groupe client
□       □□□ _layout.tsx
```

```
□   □□□ (tabs)/          # Navigation tabs client
□   □  □□□ _layout.tsx
□   □  □□□ index.tsx      # Accueil client
□   □  □□□ quotes.tsx     # Mes devis
□   □  □□□ invoices.tsx   # Mes factures
□   □  □□□ reservations.tsx # Mes réservations
□   □  □□□ messages.tsx   # Messagerie
□   □  □□□ notifications.tsx # Mes notifications
□   □  □□□ profile.tsx    # Profil
□   □  □□□ more.tsx       # Menu étendu
□   □□□ quote-detail.tsx
□   □□□ invoice-detail.tsx
□   □□□ reservation-detail.tsx
□   □□□ request-reservation.tsx
□   □□□ new-quote.tsx     # Nouvelle demande de devis
□   □□□ chat-detail.tsx   # Conversation support
□   □□□ history.tsx       # Historique complet
□   □□□ support-history.tsx
□   □□□ delete-account.tsx
□□□ components/           # Composants réutilisables
□  □□□ CustomAlert.tsx     # Modale alerte (Reanimated)
□  □□□ DateTimePicker.tsx  # Sélecteur date/heure
□  □□□ ErrorBoundary.tsx   # Boundary erreur React
□  □□□ ErrorFallback.tsx   # UI de fallback erreur
□  □□□ FilterChip.tsx      # Chips de filtre
□  □□□ FloatingSupport.tsx # Bouton support flottant
□  □□□ KeyboardAwareScrollViewCompat.tsx
□  □□□ OCRScannerModal.tsx # Scanner OCR via Gemini AI
□  □□□ SocialLoginButtons.tsx # Boutons Google/Apple Sign-In
□  □□□ StatusDropdown.tsx  # Sélecteur statut
□□□ lib/                  # Couche service / utilitaires
□  □□□ admin-api.ts        # Client API admin (avec fallback natif)
□  □□□ api.ts              # Client API client
□  □□□ auth-context.tsx    # Contexte React authentification
□  □□□ calendar.ts         # Intégration calendrier natif
□  □□□ firebase.ts         # Initialisation Firebase
□  □□□ new-client-store.ts # Store création client (zustand-like)
□  □□□ pdf-download.ts     # Téléchargement PDF
□  □□□ push-notifications.ts # Gestion notifications push
□  □□□ query-client.ts     # Client React Query configuré
□  □□□ theme.tsx           # Thème dark/light
□□□ server/               # Backend Express proxy
□  □□□ index.ts            # Point d'entrée serveur
□  □□□ routes.ts           # Toutes les routes API (2054 lignes)
□  □□□ social-auth.ts      # Routes auth sociale Firebase
□  □□□ templates/
□   □□□ landing-page.html  # Page d'accueil web statique
□□□ assets/
□  □□□ images/             # Logo, icônes
□□□ app.json               # Configuration Expo
□□□ eas.json               # Configuration EAS Build
□□□ package.json           # Dépendances npm
□□□ tsconfig.json          # Configuration TypeScript
```

5. Frontend — Routing et navigation

Système de routing

L'application utilise Expo Router v6 (file-based routing, similaire à Next.js App Router). Chaque fichier dans app/ correspond à une route.

Groupes de routes

```
(auth)   → Routes publiques (login, register, forgot-password)
(admin)  → Routes admin/employé — protégées, détectées via rôle
(main)   → Routes client — protégées, accès limité
```

Logique de redirection initiale (app/index.tsx)

```
Démarrage app
  ↓
isLoading? → Affiche splash
  ↓
isAuthenticated?
 NON → router.replace("/(auth)/login")
 OUI →
  isAdminOrEmployee? → router.replace("/(admin)/(tabs)")
  sinon              → router.replace("/(main)/(tabs)")
```

Navigation tabs Admin

Tab	Icône	Contenu
Tableau de bord		Statistiques, activité récente
Clients		CRUD clients
Devis		Liste et gestion devis
Factures		Liste et gestion factures
Réservations		Agenda réservations
Créer		Création rapide
Plus		Services, utilisateurs, paramètres

Navigation tabs Client

Tab	Icône	Contenu
Accueil	□	Résumé activité
Mes devis	□	Devis reçus
Mes factures	□	Factures reçues
Réservations	□	Mes rendez-vous
Messages	□	Support / messagerie
Notifications	□	Centre de notifications

---

6. Authentification et sécurité

Flux d'authentification

AUTHENTIFICATION CLASSIQUE (EMAIL/PASSWORD)

```
Client □ POST /api/login □ Proxy Express
                □ POST saas2.mytoolsgroup.eu/api/login
                □ { accessToken, refreshToken, user }
    □ accessToken stocké SecureStore (iOS Keychain / Android Keystore)
    □ refreshToken stocké SecureStore
```

AUTHENTIFICATION SOCIALE (GOOGLE / APPLE)

```
Client (Firebase SDK)
  □ signInWithPopup (web) / GoogleSignIn (native)
Firebase Auth (Google Cloud)
  □ idToken Firebase
Client □ POST /api/auth/social □ Proxy Express
                □ Firebase Admin SDK (vérification idToken)
                □ POST saas2/.../social-login (avec uid + email)
                □ { accessToken, refreshToken, user }
                OU { needsRegistration: true }
```

Cas particulier : Si l'utilisateur Firebase n'existe pas encore dans le SaaS, l'app affiche un formulaire d'inscription pré-rempli avec les données Firebase (email, displayName).

AUTHENTIFICATION BIOMÉTRIQUE (TOUCH ID / FACE ID)

- Disponible si l'utilisateur a activé l'option dans les paramètres
- Le token de session est stocké dans SecureStore
- LocalAuthentication.authenticateAsync() est appelé, puis le token est récupéré pour recréer la session

Stockage des tokens

Plateforme	Mécanisme	Sécurité
iOS	expo-secure-store iOS Keychain	AES-256, protégé par Secure Enclave
Android	expo-secure-store Android Keystore	AES-256, accès limité à l'app
Web	AsyncStorage (localStorage)	Non chiffré — limitation connue

Gestion des rôles

Les rôles sont détectés côté client à partir du champ user.role retourné par l'API :

```
const ADMIN_ROLES = ["admin", "super_admin", "superadmin", "root_admin", "root", "ROOT", "ROOT_ADMIN", "SUPER_ADMIN", "superAdmin"];
const EMPLOYEE_ROLES = ["employe", "employee", "manager", "EMPLOYE", "EMPLOYEE", "MANAGER"];
```

Point d'attention : La vérification des rôles est effectuée côté client. Le SaaS externe est responsable de l'application des permissions côté serveur. L'app ne fait que router vers le bon groupe de routes selon le rôle.

Refresh de token

- Un intercepteur surveille les réponses HTTP 401
- Si détecté, POST /api/refresh est appelé avec le refreshToken
- En cas d'échec du refresh, onTokenExpired() est déclenché logout automatique

---

7. Gestion d'état et données

React Query (server state)

Toutes les données issues de l'API sont gérées par @tanstack/react-query v5 :

```
// Exemple pattern utilisé
const { data, isLoading, error } = useQuery({
  queryKey: ['/api/quotes'],
  // queryFn omise — fournie par le queryClient global configuré
});

// Mutations
const mutation = useMutation({
  mutationFn: (data) => apiRequest('POST', '/api/quotes', data),
  onSuccess: () => queryClient.invalidateQueries({ queryKey: ['/api/quotes'] }),
});
```

Configuration du QueryClient (lib/query-client.ts) :

- staleTime: données fraîches pendant 5 minutes
- retry: 1 tentative sur erreur réseau
- defaultOptions.queries.queryFn : fetcher global avec gestion d'erreurs HTTP

Contexte React (local state partagé)

AuthContext (lib/auth-context.tsx) expose :

- user — profil utilisateur courant
- isAuthenticated, isAdmin, isEmployee, isAdminOrEmployee
- accessToken — token JWT courant
- login(), logout(), register(), biometricLogin(), socialLogin()

ThemeContext (lib/theme.tsx) expose :

- theme — "dark" | "light"
- colors — palette de couleurs adaptée au thème
- Toggle thème persisté via AsyncStorage

State local

- useState pour les formulaires et états UI éphémères
- new-client-store.ts — store dédié à la création de client (formulaire multi-étapes)

---

8. API Backend — Proxy Express

Rôle du proxy

Le serveur Express (server/routes.ts, 2054 lignes) agit comme proxy transparent entre l'app mobile et le SaaS externe. Il gère :

- Forwarding des requêtes vers saas2.mytoolsgroup.eu/api avec double fallback
- Authentification sociale via Firebase Admin SDK
- Uploads de photos (multipart/form-data SaaS)
- Persistance locale (PostgreSQL) pour données complémentaires
- Logs d'activité admin
- Swagger spec pour documentation API interne

Endpoints exposés

AUTHENTIFICATION

Méthode	Route	Description
POST	/api/login	Connexion email/password
POST	/api/register	Inscription
POST	/api/refresh	Rafraîchissement token
GET	/api/auth/me	Profil courant
GET	/api/auth/user	Données utilisateur
PUT	/api/auth/user	Mise à jour profil
POST	/api/auth/change-password	Changement mot de passe
DELETE	/api/users/me	Suppression compte (RGPD)
POST	/api/auth/social	Auth sociale Firebase (voir social-auth.ts)

DOCUMENTS (DEVIS & FACTURES)

Méthode	Route	Description
GET	/api/quotes	Liste des devis
GET	/api/quotes/:id	Détail devis
POST	/api/quotes/:id/accept	Accepter un devis
POST	/api/quotes/:id/reject	Refuser un devis
POST	/api/quotes/:id/create-reservation	Créer réservation depuis devis
GET	/api/invoices	Liste des factures
GET	/api/invoices/:id	Détail facture

RÉSERVATIONS

Méthode	Route	Description
GET	/api/reservations	Liste réservations
GET	/api/reservations/:id	Détail réservation
POST	/api/reservations	Créer réservation
PUT	/api/reservations/:id	Modifier réservation
POST	/api/reservations/:id/confirm	Confirmer réservation
POST	/api/reservations/:id/cancel	Annuler réservation

NOTIFICATIONS

Méthode	Route	Description
GET	/api/notifications	Liste notifications
POST	/api/notifications/:id/read	Marquer lue
POST	/api/notifications/read-all	Tout marquer lu

ADMIN

Méthode	Route	Description
GET	/api/admin/logs	Logs d'activité
GET	/api/admin/logs/export	Export logs CSV
DELETE	/api/admin/logs	Purge logs
GET	/api/admin/swagger-spec	Spec API dynamique
GET	/api/admin/reservations/:id/services	Services d'une réservation
POST	/api/admin/quotes/:docId/media	Upload photos devis
POST	/api/admin/invoices/:docId/media	Upload photos facture

DIVERS

Méthode	Route	Description
GET	/api/services	Liste services garage
GET	/api/public/garages	Garages publics
POST	/api/support/contact	Envoi message support
GET	/api/support/tickets	Tickets support
POST	/api/ocr/analyze	Analyse OCR Gemini AI

Mécanisme de fallback backend

```
const EXTERNAL_API_FALLBACKS = [
  "https://saas2.mytoolsgroup.eu/api",  // Primaire
  "https://saas3.mytoolsgroup.eu/api",  // Fallback
];

// Toute erreur réseau (ECONNREFUSED, ENOTFOUND, ETIMEDOUT)
// déclenche automatiquement le fallback vers saas3
```

Côté client natif (lib/admin-api.ts), un double fallback identique est implémenté pour accéder directement au SaaS sans passer par le proxy Replit — utilisé notamment lorsque le proxy est indisponible.

Upload de photos

Les uploads utilisent Busboy pour le parsing multipart :

- Stockage local dans /uploads/ sur le serveur Express
- Envoi au SaaS via POST {saas}/quotes/{id}/media ou {saas}/invoices/{id}/media
- Accessible via /uploads/{filename} (route statique Express)

9. Intégration Firebase

Firebase Client (lib/firebase.ts)

```
const FIREBASE_CONFIG = {
  apiKey:       "AIzaSyCCjrfHzHimlgfPjJbrWdGME_AV7jh-gdc",  // Clé web
  authDomain:   "crud-ae9d9.firebaseapp.com",
  projectId:    "crud-ae9d9",
  storageBucket: "crud-ae9d9.firebasestorage.app",
  messagingSenderId: "129808585113",
  appId:        "1:129808585113:web:03c4b7847847bf4e9b4308",  // App web
};
```

Initialisation conditionnelle : Firebase n'est initialisé que si la config est présente et valide. Sur les plateformes mobiles, getReactNativePersistence(AsyncStorage) est utilisé pour la persistance de session.

Google Sign-In

Plateforme	Mécanisme	Client ID
Web	signInWithPopup Firebase	129808585113-
iOS natif	Scheme inversé	129808585113-
Android natif	Intent Google Sign-In	129808585113-
Expo Go	Message informatif (non supporté)	—

Apple Sign-In

- Utilise expo-apple-authentication (AppleAuthentication.signInAsync)
- Retourne un identityToken envoyé au proxy Firebase Admin SDK le vérifie
- Apple Key ID : L86N86P8ZD, Team ID : GP593F562X

Firebase Admin SDK (server/social-auth.ts)

- Initialisé avec FIREBASESERVICEACCOUNT_JSON (secret Replit)
- Vérifie les idToken reçus du client : admin.auth().verifyIdToken(idToken)
- Extrait uid, email, displayName pour les passer au SaaS

10. Base de données locale (PostgreSQL)

Le proxy Express maintient une base PostgreSQL (Replit DB) avec 4 tables locales :

Schéma

```
-- Comptes supprimés (conformité RGPD)
CREATE TABLE deleted_accounts (
  id            SERIAL PRIMARY KEY,
  external_user_id TEXT,
  email         TEXT,
  user_data     JSONB,
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Montants locaux des documents (calculs côté app)
CREATE TABLE document_amounts (
  id              SERIAL PRIMARY KEY,
  doc_id          TEXT NOT NULL UNIQUE,
  doc_type        TEXT NOT NULL,      -- 'quote' | 'invoice'
  price_excluding_tax NUMERIC,
  total_including_tax NUMERIC,
  tax_amount      NUMERIC,
  items           JSONB,
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Photos des documents
CREATE TABLE document_photos (
  id      SERIAL PRIMARY KEY,
  doc_id  TEXT NOT NULL,
  doc_type TEXT NOT NULL,
  photo_uri TEXT NOT NULL,
  ...
);

-- Logs d'activité admin
CREATE TABLE admin_activity_logs (
  ...
);
```

Remarque : Les données métier (devis, factures, clients, réservations) ne sont pas stockées localement — elles résident exclusivement sur le SaaS externe.

---

11. Notifications push

Architecture

App expo-notifications Expo Push Service

> □ APNs (iOS)
> □ FCM (Android)

Fichier : lib/push-notifications.ts

Flux d'enregistrement

- Au login, registerForPushNotificationsAsync() est appelé
- Permission notification demandée à l'utilisateur
- Token Expo Push obtenu et envoyé au SaaS : POST /api/notifications/push-token
- Sur web : Web Push API utilisée avec VAPID

Polling

Pour les notifications en temps réel, un polling toutes les 30 secondes est implémenté via startNotificationPolling() (appel GET /api/notifications). Aucun WebSocket utilisé pour les notifications.

---

12. Build et déploiement (EAS)

Configuration EAS (eas.json)

Profil	Distribution	Plateforme	Auto-incrément	Usage
development	internal	iOS APK + Android APK	□	Développement
preview	internal	iOS Simulator + Android APK	□	Tests internes
production	store	iOS App Store + Android Bundle	□	Production

Identifiants de build

Champ	Valeur
Bundle ID (iOS)	app.mytoolsmobile.mytoolsgroup.eu
Package (Android)	app.mytoolsmobile.mytoolsgroup.eu
EAS Project ID	2429ee3a-9dd5-4767-9532-175f1db29ff3
EAS Owner	mytoolsgroup
Build Number actuel	21+ (géré par EAS remote)
App Store ID	6759137046
Apple Team ID	GP593F562X

Credentials iOS

- Type : EAS Managed (credentialsSource: "remote")
- Certificat : Distribution Certificate créé et géré automatiquement par EAS
- Provisioning Profile : App Store Distribution, géré par EAS
- APNs Key : AuthKey_L86N86P8ZD.p8 (Key ID : L86N86P8ZD)

Source de version

appVersionSource: "remote" — la version est gérée exclusivement par EAS. Le champ buildNumber dans app.json est ignoré à l'exécution du build.

Authentification CI

EXPO_TOKEN est configuré comme secret Replit pour permettre les builds EAS en mode non-interactif.

---

13. Configuration des environnements

Variables d'environnement

Variable	Portée	Valeur dev	Description
EXPOPUBLICDOMAIN	Frontend	{replit-domain}:5000	URL du proxy backend
EXPOPUBLICFIREBASEAPIKEY	Frontend	AIzaSyCC...	Clé web Firebase
EXPOPUBLICFIREBASEAPPID	Frontend	1:129808...	App ID Firebase web
EXPOPUBLICFIREBASEAUTHDOMAIN	Frontend	crud-ae9d9.firebaseapp.com	Auth domain
EXPOPUBLICFIREBASEPROJECTID	Frontend	crud-ae9d9	Project Firebase
EXPOPUBLICGOOGLEWEBCLIENT_ID	Frontend	129808...android...	Client ID Google
EXPOPUBLICGOOGLEIOSCLIENT_ID	Frontend	129808...q81u...	Client ID Google iOS
FIREBASESERVICEACCOUNT_JSON	Backend	Secret Replit	Credentials Firebase Admin
DATABASE_URL	Backend	Secret Replit	URL PostgreSQL
EXPO_TOKEN	CI	Secret Replit	Token EAS CLI

Environnements de build EAS

Les variables EXPOPUBLIC* sont dupliquées dans la section env de chaque profil EAS (eas.json) pour être disponibles au moment du build.

---

14. Composants partagés

CustomAlert.tsx

Remplacement natif de Alert.alert() avec animation Reanimated v4.

- useSharedValue, useAnimatedStyle, withSpring pour l'animation d'entrée
- Supporte : titre, message, boutons multiples, styles default / destructive / cancel

OCRScannerModal.tsx

Scanner OCR utilisant Gemini AI (@google/genai) :

- Capture photo via expo-camera
- Encode en base64
- Envoie à POST /api/ocr/analyze Gemini Pro Vision
- Extrait données structurées (immatriculation, modèle, kilométrage, etc.)

SocialLoginButtons.tsx

Gestion complète des connexions sociales :

- Web : signInWithPopup Firebase
- iOS natif : Google Sign-In via scheme inversé + Apple Sign-In via expo-apple-authentication
- Expo Go : Message informatif (limitation Expo Go)

ErrorBoundary.tsx + ErrorFallback.tsx

- ErrorBoundary : class component React avec getDerivedStateFromError + componentDidCatch
- ErrorFallback : composant fonctionnel avec reloadAppAsync() pour redémarrer l'app
- Utilise useColorScheme et useSafeAreaInsets pour un affichage adapté

FloatingSupport.tsx

Bouton de support flottant accessible depuis toutes les pages avec deeplink vers le chat support.

---

15. Points d'attention pour l'audit

Architecture & conception

- Gestion des rôles côté client uniquement : La vérification des permissions est faite dans detectIsAdmin() / detectIsEmployee() côté app. Le SaaS externe doit impérativement appliquer ses propres contrôles d'accès côté serveur.
- Double client HTTP : lib/api.ts (clients) et lib/admin-api.ts (admin) ont des logiques de retry/fallback similaires mais séparées — refactorisation potentielle.

- Proxy obligatoire : L'architecture impose de passer par le proxy Replit en production web, créant un SPOF. En natif, le double fallback direct (saas2 □ saas3) contourne ce risque.
- Stockage web non sécurisé : Sur plateforme web, les tokens JWT sont stockés dans AsyncStorage (localStorage) — non chiffré. Acceptable pour une app interne, à documenter dans la politique de sécurité.

Dépendances

- New Architecture activée (newArchEnabled: true) : Requis par Expo SDK 54 + React Native 0.81.5 + Reanimated v4. Toutes les bibliothèques majeures du projet ont été vérifiées comme compatibles.
- expo-glass-effect v0.1.9 : Bibliothèque expérimentale (tab bar iOS 26 "liquid glass"). Dispose d'un fallback isLiquidGlassAvailable() pour les versions antérieures.
- react-native-worklets v0.5.1 : Peer dependency de Reanimated v4, architecture worklets séparée. Version récente mais stable.
- expo-doctor : 17/17 checks □ — aucune incompatibilité de dépendances détectée.

Sécurité

- Firebase API Key exposée : La EXPOPUBLICFIREBASEAPIKEY est visible dans le bundle client (comportement normal pour les apps Firebase). La sécurité est assurée par les règles Firebase et les domaines autorisés, non par le secret de la clé.
- FIREBASESERVICEACCOUNT_JSON : Correctement stocké comme secret Replit, injecté uniquement côté serveur (server/social-auth.ts). Ne circule jamais côté client.
- Aucune donnée sensible dans le code source : Les tokens, mots de passe et credentials sont gérés via les secrets Replit / EAS — non commités.

Performance

- Polling notifications : Interrogation toutes les 30s au lieu de WebSocket — acceptable pour les volumes actuels, à réévaluer à grande échelle.
- Taille du bundle server/routes.ts : 2054 lignes dans un fichier unique — refactorisation en modules recommandée pour la maintenabilité.

Annexe A — Flux d'authentification complet

```
□□□□□□□□□□□□□   □□□□□□□□□□□□□□□□□□   □□□□□□□□□□□□□□□□□□□   □□□□□□□□□□□□□
□ Client □   □ Proxy Express□   □ Firebase Admin□   □ SaaS ext □
□□□□□□□□□□□□□   □□□□□□□□□□□□□□□□□□   □□□□□□□□□□□□□□□□□□□   □□□□□□□□□□□□□
  □           □             □            □
  □□□POST /api/auth/social □□□□□□□□□□□□□□□□>            □
  □  { idToken, provider }          □             □
  □           □□□verifyIdToken()□□>□            □
  □           □<□□ { uid, email }□□□            □
  □           □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□POST /social-login□□>
  □           □<□□□□□□□□□□□□□□□□□□□□□□□□□□□ { accessToken, refreshToken, user }
  □<□□ { accessToken, refreshToken, user }            □
```

```
□          □          □          □
□ [stocke tokens SecureStore]      □          □
□          □          □          □
```

Annexe B — Gestion des erreurs réseau

```
fetchWithNativeFallback(endpoint)
  □
fetchWithTimeout(url, 15s)
  □ Timeout / Erreur réseau
fetchWithRetry(url, retries=1)
  □ Toujours en erreur
Fallback saas2 □ saas3
  □ Toujours en erreur
throw NetworkError (message FR pour l'utilisateur)
```

Annexe C — Variables d'environnement requises

Pour déployer une nouvelle instance du projet, les secrets suivants sont nécessaires :

Secret	Obligatoire	Description
FIREBASESERVICEACCOUNT_JSON	□	JSON complet du compte de service Firebase
DATABASE_URL	□	URL de connexion PostgreSQL
EXPO_TOKEN	□	Token EAS CLI (expo.dev)

Document généré pour audit de code — MyToolsApp v2.0.1 — Mars 2026